

هوش مصنوعی و کارگاه

دانشکده ریاضی و علوم کامپیوتر دانشگاه صنعتی امیرکبیر



مدرس کلاس: مهدی قطعی

استاد گروه علوم کامپیوتر

www.aut.ac.ir

مدرس کارگاه: بهنام یوسفی مهر

دانشجوی دکتری علوم کامپیوتر

Artificial Intelligence & Workshop

- Associative Memories and Hopfield Networks

Associative Memories and Hopfield Networks

- Optimization for Neural Networks
 - Training
 - **The delta rule (Adaline,...)**
 - Gradient-Based Methods
 - Stochastic Gradient-Based Methods (ADAM, ...)
- Neural Networks for Optimization
 - Associative Memory
 - Hopfield Neural Networks

Associative Memory

x	1	-1	1	1	-1	-1	-1	1
---	---	----	---	---	----	----	----	---

y
1
-1
1

$A = y \cdot x^t$	1	-1	1	1	-1	-1	-1	1
	-1	1	-1	-1	1	1	1	-1
	1	-1	1	1	-1	-1	-1	1

$A \cdot x$	hardlimit	
	8	1
	-8	-1
	8	1

Associative Memory

x	1	-1	1	1	-1	-1	-1	1
	-1	1	-1	-1	-1	1	1	1

y								
1	-1							
-1	-1							
1	-1							
A1=y*x^t	1	-1	1	1	-1	-1	-1	1
	-1	1	-1	-1	1	1	1	-1
	1	-1	1	1	-1	-1	-1	1
A2=y*x^t	1	-1	1	1	1	-1	-1	-1
	1	-1	1	1	1	-1	-1	-1
	1	-1	1	1	1	-1	-1	-1
A=A1+A2	2	-2	2	2	0	-2	-2	0
	0	0	0	0	2	0	0	-2
	2	-2	2	2	0	-2	-2	0

	hardlimit	
A*x1	12	1
	-4	-1
	12	1

A*x2	-12	-1
	-4	-1
	-12	-1

Iterative Autoassociative Networks

- Example:

$$x = (1, 1, 1, -1) \quad W = \begin{bmatrix} 0 & 1 & 1 & -1 \\ 1 & 0 & 1 & -1 \\ 1 & 1 & 0 & -1 \\ -1 & -1 & -1 & 0 \end{bmatrix}$$

Output units are
threshold units

An incomplete recall input : $x' = (1, 0, 0, 0)$

$$Wx' = (0, 1, 1, -1) = x''$$

$$Wx'' = (3, 2, 2, -2) \rightarrow (1, 1, 1, -1) = x$$

- In general: using current output as input of the next iteration

$x(0)$ = initial recall input

$$x(I) = S(Wx(I-1)), \quad I = 1, 2, \dots$$

until $x(N) = x(K)$ for some $K < N$

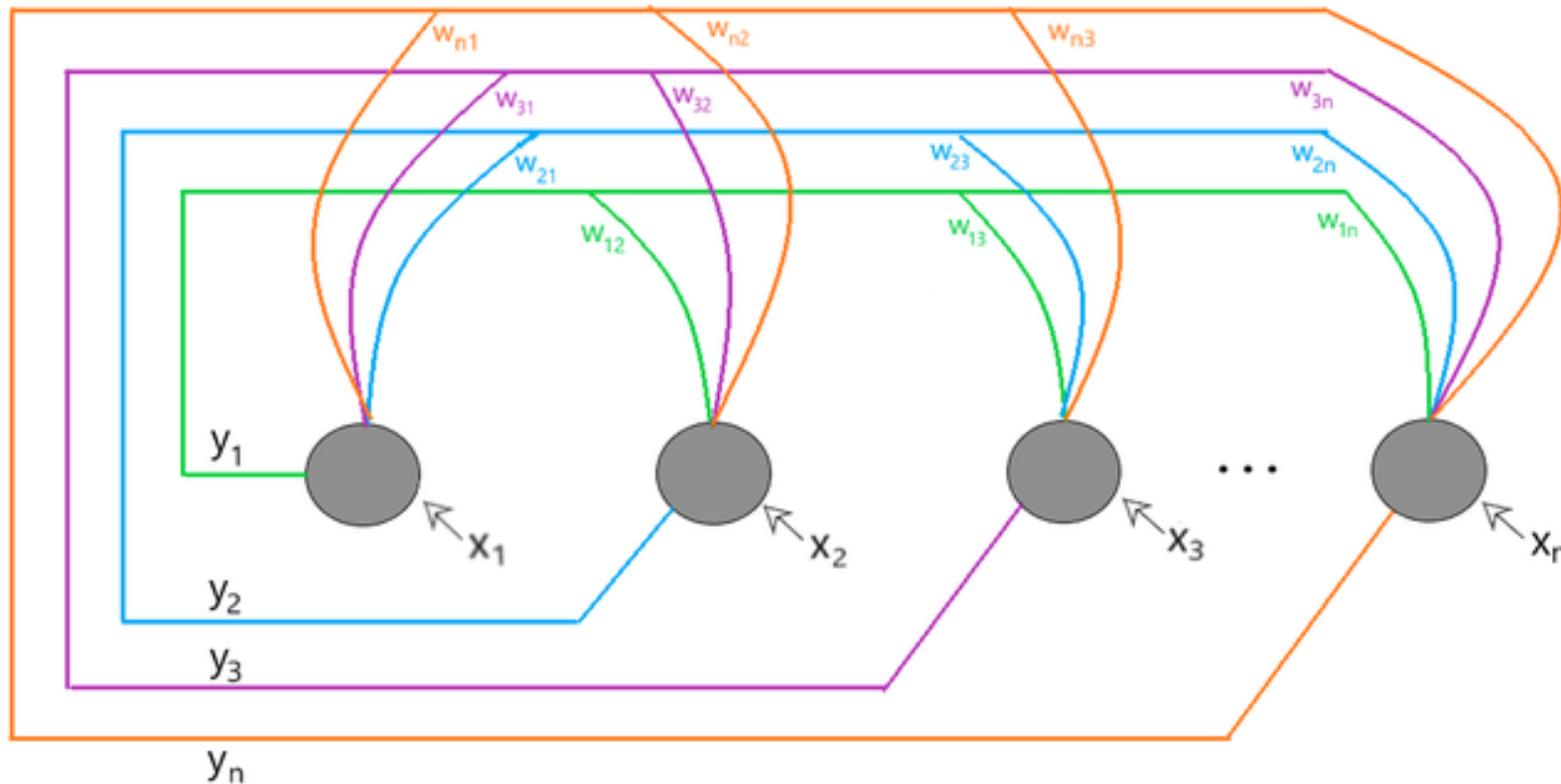
Genuine memory & Spurious memory



- Dynamic System: state vector $x(I)$
 - If $K = N-1$, $x(N)$ is a stable state (fixed point)
$$f(Wx(N)) = f(Wx(N-1)) = x(N)$$
 - If $x(K)$ is one of the stored pattern, then $x(K)$ is called a **genuine memory**
 - Otherwise, $x(K)$ is a **spurious memory** (caused by cross-talk/interference between genuine memories)
 - Each fixed point (genuine or spurious memory) is an **attractor** (with different attraction basin)

Hopfield Models

- A single layer network with full connection
 - each node as both input and output units
 - node values are iteratively updated, based on the weighted inputs from all other nodes, until stabilized



Hopfield Models

- More than an AM
 - Other applications e.g., combinatorial optimization
- Different forms: discrete & continuous
- Major contribution of John Hopfield to NN
 - Treating a network as a dynamic system
 - Introduced the notion of **energy function** and attractors into NN research



He received a Ph.D. in physics from Cornell University in 1958, and subsequently was a faculty member at University of California, Berkeley (physics), Princeton University (physics), California Institute of Technology (Chemistry and Biology) and again at Princeton.

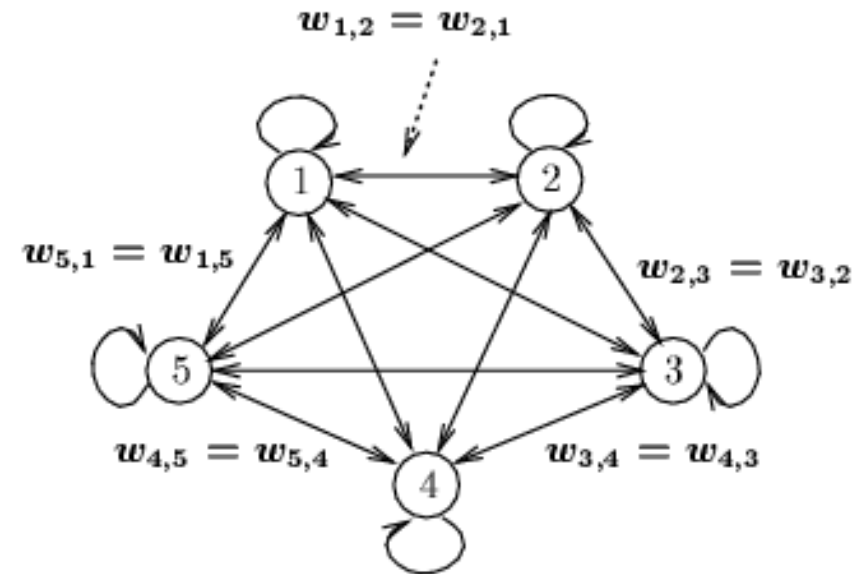
Discrete Hopfield Model (DHM) as AM

■ Architecture:

- single layer (units serve as both input and output)
- nodes are threshold units (binary or bipolar)
- Weights: fully connected, symmetric, and zero diagonal

$$w_{ij} = w_{ji}$$
$$w_{ii} \geq 0$$

- External inputs may be transient or permanent



Discrete Hopfield Model (DHM) as AM

NOTATION:

$T = \{i_1, \dots, i_P\}$: training set.

$x_{p,j}(t)$: value generated by the j th node at time t , for p th input pattern.

$w_{k,j}$: weight of connection from j th to k th node.

$I_{p,k}$: external input to k th node, for p th input vector (includes threshold term).

The node output function is described by:

$$x_{p,k}(t+1) = \text{sgn} \left(\sum_{j=1}^n w_{k,j} x_{p,j}(t) + I_{p,k} \right)$$

■ Weights:

- To store patterns i_p , $p = 1, 2, \dots, P$

bipolar:

$$w_{jk} = \frac{1}{P} \sum_p i_{p,k} i_{p,j} \quad k \neq j$$
$$w_{kk} = 0$$

same as Hebbian rule (with zero diagonal)

binary:

$$w_{jk} = \sum_p (2i_{p,k} - 1)(2i_{p,j} - 1) \quad k \neq j$$
$$w_{kk} = 0$$

converting i_p to bipolar when constructing W .

■ Recall

- Use an input vector to recall a stored vector
- Each time, **randomly** select a unit for update
- Periodically check for convergence

Example 1

$$w_{jk} = \frac{1}{P} \sum_p i_{p,k} i_{p,j} \quad k \neq j$$

$$w_{kk} = 0$$

$$I = [$$

$$\begin{array}{cccccc} 1 & -1 & 1 & -1 & -1 & -1 \\ 1 & -1 & -1 & -1 & 1 & 1 \\ -1 & -1 & 1 & 1 & 1 & 1 \\ 1 & 1 & -1 & -1 & 1 & 1 \\ -1 & -1 & 1 & 1 & -1 & -1 \end{array}]$$

$$W = \begin{bmatrix} 0 & 0.2000 & -0.6000 & -1.0000 & 0.2000 & 0.2000 \\ 0.2000 & 0 & -0.6000 & -0.2000 & 0.2000 & 0.2000 \\ -0.6000 & -0.6000 & 0 & 0.6000 & -0.6000 & -0.6000 \\ -1.0000 & -0.2000 & 0.6000 & 0 & -0.2000 & -0.2000 \\ 0.2000 & 0.2000 & -0.6000 & -0.2000 & 0 & 1.0000 \\ 0.2000 & 0.2000 & -0.6000 & -0.2000 & 1.0000 & 0 \end{bmatrix}$$

$$X(1) = [-1 \quad 1 \quad 1 \quad -1 \quad -1 \quad 1]$$

$$b = [-0.1115 \quad -0.1719 \quad -1.8816 \quad -0.3732 \quad -1.5464 \quad 0.3231]$$

$$X(5) = [1 \quad 1 \quad -1 \quad -1 \quad 1 \quad 1]$$

$$X(1) = [1 \quad -1 \quad -1 \quad 1 \quad -1 \quad 1]$$

$$b = [-1.1178 \quad -0.4937 \quad 0.8182 \quad 0.7001 \quad -0.8393 \quad 0.7178]$$

$$X(7) = [-1 \quad -1 \quad 1 \quad 1 \quad -1 \quad -1]$$

■ Notes:

- 1.Theoretically, to guarantee convergence of the recall process (avoid oscillation), only one unit is allowed to update its activation at a time during the computation (**asynchronous model**).
- 2.However, the system may converge faster if all units are allowed to update their activations at the same time (synchronous model).
- 3.Each unit should have equal probability to be selected
- 4.Convergence test:

$$x_k(\textit{previous}) = x_k(\textit{current}) \quad \forall k$$

Example 2:

- A 4 node network, stores 2 patterns (1 1 1 1) and (-1 -1 -1 -1)
- Weights:

$$w_{\ell,j} = 1, \text{ for } \ell \neq j, \text{ and } w_{j,j} = 0 \text{ for all } j$$

- Corrupted input pattern: (1 1 1 -1)

Node selection	output pattern
node 2: $w_{2,1}x_1 + w_{2,3}x_3 + w_{2,4}x_4 + I_2 = 1 + 1 - 1 + 1 = 2$	(1 1 -1)
node 4: $1 + 1 + 1 - 1 = 2$	(1 1 1 1)

No more change of state will occur, the correct pattern is recovered

- Equaldistance: (1 1 -1 -1)

node 2: net = 0, no change	(1 1 -1 -1)
node 3: net = 0, change state from -1 to 1	(1 1 1 -1)
node 4: net = 0, change state from -1 to 1	(1 1 1 1)

No more change of state will occur, the correct pattern is recovered

If a different tie breaker is used (if input = 0, output -1), the stored pattern (-1 -1 -1 -1) will be recalled
 In more complicated situations, different order of node selections may lead the system to converge to different attractors.

Example 3:

- Missing input element: (1 0 -1 -1)

Node
selection

output
pattern

node 2: $w_{12}x_1 + w_{32}x_3 + w_{42}x_4 + I_2 = 1 - 1 - 1 + 0 < 0$ (1 -1 -1 -1)

node 1: net = -3, change state to -1 (-1 -1 -1 -1)

No more change of state will occur, the correct pattern is recovered

- Missing input element: (0 0 0 -1)

the correct pattern (-1 -1 -1 -1) is recovered

This is because the AM has only 2 attractors

(1 1 1 1) and (-1 -1 -1 -1)

When spurious attractors exist (with more memories), pattern completion may be incorrect (been attracted to wrong, spurious attractors).

Example 4:

- A 4 node network, stores 3 patterns
(1 1 -1 -1), (1 1 1 1) and (-1 -1 1 1)

- Weights:

$$\begin{pmatrix} 0 & 1 & -1/3 & -1/3 \\ 1 & 0 & -1/3 & -1/3 \\ -1/3 & -1/3 & 0 & 1 \\ -1/3 & -1/3 & 1 & 0 \end{pmatrix}$$

- Corrupted input pattern: (-1 -1 -1 -1)

- If node 4 is selected:

$$(-1/3 \ -1/3 \ 1 \ 0) \ (-1 \ -1 \ -1 \ -1) + (-1) = 1/3 + 1/3 -1 -1 = -4/3,$$

- No change of state for node 4
- Same for all other nodes, net stabilized at (-1 -1 -1 -1)
- A spurious state/attractor is recalled

Example 4:

- For input pattern $(-1 -1 -1 0)$
 - If node 4 is selected first,
 $(-1/3 -1/3 1 0) (-1 -1 -1 0) + (0) = 1/3 + 1/3 - 1 - 0 - 0 = -1/3$,
change state to -1, then same as in the previous example,
network stabilized at $(-1 -1 -1 -1)$
 - If the node 3 is selected before 4 and if the input is transient, the net will
stabilized at state $(-1 -1 1 1)$, a genuine attractor

$$W = \begin{pmatrix} 0 & 1 & -1/3 & -1/3 \\ 1 & 0 & -1/3 & -1/3 \\ -1/3 & -1/3 & 0 & 1 \\ -1/3 & -1/3 & 1 & 0 \end{pmatrix}$$

Convergence Analysis of DHM

- **Two questions:**

1. Will Hopfield AM converge (stop) with **any** given recall input?
2. Will Hopfield AM converge to the stored pattern that is **closest** to the recall input ?

- **Hopfield provides answer to the first question**

- By introducing an **energy function** to this model,
- No satisfactory answer to the second question so far.

- **Energy function:**

- Notion in thermo-dynamic physical systems. The system has a tendency to move toward lower energy state.
- Also known as **Lyapunov** function in mathematics. After Lyapunov theorem for the stability of a system of differential equations.

Convergence Analysis of DHM

- In general, the energy function $E(x(t))$, where $x(t)$ is the state of the system at step (time) t , must satisfy two conditions

1. $E(t)$ is bounded from below

$$E(t) \geq c \quad \forall t$$

2. $E(t)$ is monotonically non-increasing with time.

$$\Delta E(t+1) = E(t+1) - E(t) \leq 0 \quad (\text{in continuous version : } \dot{E}(t) \leq 0)$$

- Therefore, if the system's state change is associated with such an energy function, its energy will continuously be reduced until it reaches a state with a (locally) minimum energy
- Each (locally) minimum energy states are attractors
- Hopfield shows his model has such an energy function, the memories (patterns) stored in DHM are attractors (other attractors are spurious)

- The energy function for DHM:
 - Assume the input vector is close to one of the attractors
-
- If $w_{\ell,j}$ is positive and large, then we expect that the ℓ th and j th nodes are frequently ON or OFF together in the attractor patterns, i.e., $\sum_{p=1}^P (i_{p,\ell} i_{p,j})$ is positive and large.
 - Similarly, $w_{\ell,j}$ is negative and large when the ℓ th and j th nodes frequently have opposite activation values for different attractor patterns, i.e., $\sum_{p=1}^P (i_{p,\ell} i_{p,j})$ is negative and large.
 - So $w_{\ell,j}$ should be proportional to $\sum_{p=1}^P (i_{p,\ell} i_{p,j})$. For strong positive or negative correlation, $w_{\ell,j}$ and $\sum_p (i_{p,\ell} i_{p,j})$ have the same sign, hence $\sum_{p=1}^P w_{\ell,j} (i_{p,\ell} i_{p,j}) > 0$.

- Self-excitation coefficient $w_{j,j} \geq 0$, often $=0$.
- Summing over all pairs of nodes, $\sum_{\ell} \sum_j w_{\ell,j} i_{p,\ell} i_{p,j}$ is positive and large for input vectors almost identical to some attractor.
- We expect that node correlations present in the attractors are absent in an input vector i distant from all attractor patterns, so that $\sum_{\ell} \sum_j w_{\ell,j} i_{\ell} i_j$ is then low or negative.
- Therefore, the energy function contains a term

$$- \left(\sum_{\ell} \sum_j w_{\ell,j} x_{\ell} x_j \right).$$

When this is minimized, the final values of various node outputs are expected to correspond to an attractor pattern.

- Network output should be close to the input vector;
For external inputs I_ℓ , another
term $-\sum_\ell I_\ell x_\ell$ is included in the energy expression;
 $I_\ell x_\ell > 0$ iff input=output for the ℓ th node.

Combining the two terms, the following “energy” or
Lyapunov function must be minimized by modifying x_ℓ
values;

$$E = -a \sum_\ell \sum_j w_{\ell,j} x_\ell x_j - b \sum_\ell I_\ell x_\ell \quad \text{where } a, b \geq 0.$$

often with $a = 1/2$, $b = 1$

Since all values in E are finite, E is finite and thus bounded from below

■ Convergence

- let k^{th} node is updated at time t , the system energy change is

$$\begin{aligned}\Delta E(t) &= E(t+1) - E(t) \\ &= -a \sum_{\ell} \sum_{j \neq \ell} w_{\ell,j} [x_{\ell}(t+1)x_j(t+1) - x_{\ell}(t)x_j(t)] \\ &\quad -b \sum_{\ell} I_{\ell} (x_{\ell}(t+1) - x_{\ell}(t)) \\ &= -a \sum_{j \neq k} ((w_{k,j} + w_{j,k}) (x_k(t+1) - x_k(t)) x_j(t)) \\ &\quad -b I_k (x_k(t+1) - x_k(t)),\end{aligned}$$

because $x_j(t+1) = x_j(t)$ for every node ($j \neq k$) not selected for updating at this step. Hence,

$$\Delta E(t) = - \left[a \sum_{j \neq k} (w_{k,j} + w_{j,k}) x_j(t) + b I_k \right] (x_k(t+1) - x_k(t))$$

- When choosing $a = 1/2$, $b = 1$

$$\begin{aligned}\Delta E(t) &= - \left(\sum_{j \neq k} w_{j,k} x_j(t) + I_k \right) (x_k(t+1) - x_k(t)) \\ &= -\text{net}_k(t) \Delta x_k(t),\end{aligned}$$

Since state change (from -1 to +1 or +1 to -1) depends on if $\text{net}_k \geq 0$ or < 0 , we have $\text{net}_k(t) \cdot \Delta x_k(t) > 0$

And therefore, $\Delta E(t) < 0$

Continuous Hopfield Model (CHM)

- Different (the original) formulation than the text
- Architecture:
 - Continuous node output, and continuous time
 - Fully connected with symmetric weights $w_{ij} = w_{ji}, w_{ii} = 0$
 - Internal activation

$$u_i : \text{ with } \frac{du_i(t)}{dt} = \sum_{j=1}^n w_{ij} x_j(t) + \theta_i = \text{net}_i(t)$$

- Output (state) $x_i(t) = f(u_i(t))$

where f is a sigmoid function to ensure binary/bipolar output.

E.g. for bipolar, use hyperbolic tangent function

$$f(x) = \tanh(x) = \frac{e^x - e^{-x}}{e^x + e^{-x}}$$

Continuous Hopfield Model (CHM)

- Computation: all units change their output (states) at the same time, based on states of all others.

- Compute net: $net_i(t) = \sum_{j=1}^n w_{ij} x_j(t) + \theta_i$
- Compute internal activation $u_i(t)$ by first-order Taylor expansion

$$u_i(t + \delta) = \int in_i(t) dt \approx u_i(t) + \frac{du_i(t)}{dt} \cdot \delta = u_i(t) + net_i \cdot \delta$$

- Compute output

$$x_i(t) = f(u_i(t))$$

Convergence:

- define an energy function,
- show that if the state update rule is followed, the system's energy always decreasing by showing derivative $dE / dt \leq 0$

$$E = -\frac{1}{2} \sum_{ij} x_i w_{ij} x_j - \sum_i \theta_i x_i$$

$$\frac{dE}{dt} = \sum_i \frac{\partial E}{\partial x_i} \cdot \frac{dx_i}{du_i} \cdot \frac{du_i}{dt} \quad (\text{chain rule, } x \text{ depends on } u, u \text{ on } t)$$

$$\frac{\partial E}{\partial x_i} = -\sum_{j \neq i} w_{j,i} x_j - \theta_i = -net_i \quad (x_i \text{ appears twice in } E)$$

$$\frac{dx_i}{du_i} = f'(u_i) > 0,$$

$$\frac{du_i}{dt} = net_i = -\frac{\partial E}{\partial x_i}$$

$$\text{Therefore, } \frac{dE}{dt} = -\sum_i f'(u_i)(net_i)^2 \leq 0$$

-
- $\dot{E}$ asymptotically approaches zero when $f'(u_i)$ approaches 1 or 0 (-1 for bipolar) for all i .
 - The system reaches a local minimum energy state
 - Gradient descent: $du_i / dt = -\partial E / \partial x_i$
 - Instead of jumping from corner to corner in a hypercube as the discrete HM does, the system of continuous HM moves in the interior of the hypercube along the gradient descent trajectory of the energy function to a local minimum energy state.

Optimization with Neural Networks

Introduction

An optimization problem consists of two parts: **Cost function** and **Constraints**

- **Constrained**

- The constraints are built in the cost function, so minimizing the cost function also satisfies the constraints

- **Unconstraint**

- There is no constraint for the problem!

- **Combinatorial**

- We separate the constraints and the cost function, minimize each of them and then add them together

Application

Applications in many fields like:

- ☐ Routing in computer networks
- ☐ VLSI circuit design
- ☐ Planning in operational and logistic systems
- ☐ Power distribution systems
- ☐ Wireless and satellite communication systems

Basic idea

- If $X_1, X_2, \dots, X_n$: decision variables
- Suppose $F(X_1, X_2, \dots, X_n)$ is our objective function .
- Constraints can be expressed as nonnegative penalty functions $C_i(X_1, X_2, \dots, X_n) = 0$ that only when $X_1, X_2, \dots, X_n$ represent a feasible solution
- By combining the penalty functions with F , the original constrained problem may be reformulated as unconstrained problem in which the goal is to minimize the quantity :

$$F^* = F(X_1, X_2, \dots, X_n) + \alpha \sum_{k=1}^m C_k(X_1, X_2, \dots, X_n)$$

TSP

- Is simple to state but very difficult to solve.
- The problem is to find the shortest possible tour through a set of N vertices so that each vertex is visited exactly once.
- This problem is known to be NP-complete

Why neural network?

- Drawbacks of conventional computing systems:
 - Perform poorly on complex problems
 - Lack the computational power
 - Don't utilize the inherent parallelism of problems
- Advantages of artificial neural networks:
 - Perform well even on complex problems
 - Very fast computational cycles if implemented in hardware
 - Can take the advantage of inherent parallelism of problems

Mapping TSP to Hopfield neural net

- ❑ There is a connection between each pair of units
- ❑ The signal sent along a connection from i to j is equal to the weight T_{ij} if i is activated. It is equal to 0 otherwise.
- ❑ A negative weight defines inhibitory connection between the two units
- ❑ It is unlikely that two units with negative weight will be active or “on” at the same time

Discrete Hopfield Model

- connection weights are not learned
- Hopfield network evolves by updating the activation of each unit in turn
- In final state, all units are stable according to the update rule
- The units are updated at random, one unit at a time

set V_i to 0 if $\sum_j T_{ij} V_j < \theta_i$

set V_i to 1 if $\sum_j T_{ij} V_j > \theta_i$

do not change V_i if $\sum_j T_{ij} V_j = \theta_i$,

$\{V_i\}_{i=1,\dots,L}$,

L : number of units

V_i : activation level of unit i

T_{ij} : connection weight between units i and j

θ_i : threshold of unit i .

Discrete Hopfield Model (Cont.)

- Energy function

$$E = -1/2 \sum_i \sum_j T_{ij} V_i V_j + \sum_i \theta_i V_i .$$

- Units changes its activation level if and only if the energy of the network decreases by doing so:

$$E_i = - \sum_j T_{ij} V_i V_j + \theta_i V_i .$$

if $V_i = 1$ then $E_i = - \sum_j T_{ij} V_j + \theta_i$
if $V_i = 0$ then $E_i = 0$.

$$\Delta E_i = - \Delta V_i (\sum_j T_{ij} V_j - \theta_i) .$$

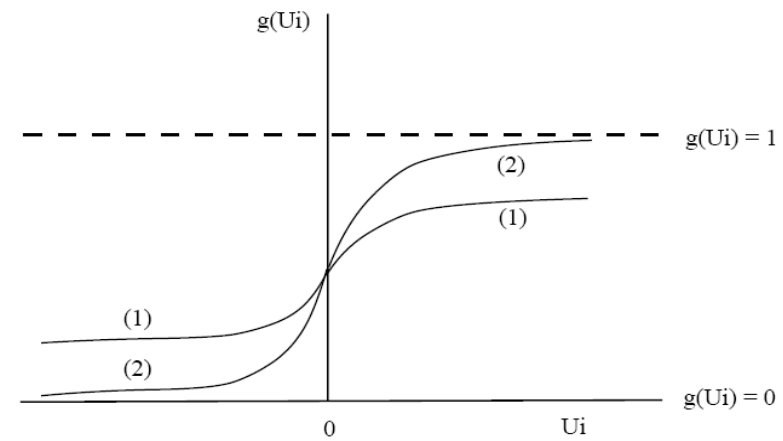
- Since the energy can only decrease over time and the number configuration is finite
the network must converge (but not necessarily the minimum energy state)

Continuous Hopfield-Tank

- Neuron function is continuous (Sigmoid function)
- The evolution of the units over time is now characterized by the following differential equation :

$$dU_i/dt = \sum_j T_{ij}V_j + I_i - U_i, \quad i=1,\dots,L$$

U_i , I_i and V_i are the input, input bias, and activation level of unit i , respectively



$$V_i = g(U_i) = \frac{1}{2} (1 + \tanh U_i/U_0) = \frac{1}{1 + e^{-2U_i/U_0}} .$$

Continuous Hopfield-Tank

- Energy function

$$E = -1/2 \sum_i \sum_j T_{ij} V_i V_j - \sum_i V_i I_i + \sum_i \int_0^{V_i} g^{-1}(x) dx .$$

$$U_i = \sum_j T_{ij} V_j + I_i .$$

$$dU_i/dt = -dE/dV_i$$

- Discrete time approximation is applied to

$$\{U_i(t+\Delta t) - U_i(t)\} / \Delta t = \sum_j T_{ij} V_j(t) + I_i - U_i(t)$$

Application of the Hopfield-Tank Model to the TSP

- (1) Choose a representation scheme which allows the activation levels of the units to be decoded into a solution of the problem.
- (2) Design an energy function whose minimum corresponds to the best solution of the problem.
- (3) Derive the connectivity of the network from the energy function.
- (4) Set up the initial activation levels of the units.

Application of the Hopfield-Tank model to the TSP

(1) The TSP is represented as an $N \times N$ matrix

(2) Energy function

$$\begin{aligned} E = & A/2 \left(\sum_X \sum_i \sum_{j \neq i} V_{Xi} V_{Xj} \right) \\ & + B/2 \left(\sum_i \sum_X \sum_{Y \neq X} V_{Xi} V_{Yi} \right) \\ & + C/2 \left(\sum_X \sum_i V_{Xi} - N \right)^2 \\ & + D/2 \left(\sum_X \sum_{Y \neq X} \sum_i d_{XY} V_{Xi} (V_{Yi+1} + V_{Yi-1}) \right), \quad (1.5) \end{aligned}$$

$$E = -1/2 \sum_{Xi} \sum_{Yj} T_{XiYj} V_{Xi} V_{Yj} - \sum_{Xi} V_{Xi} I_{Xi}. \quad (1.6)$$

(3) Bias and connection weights are derived

Application of the Hopfield-Tank model to the TSP

$$\begin{aligned} T_{XiYj} = & - A \delta_{XY} (1 - \delta_{ij}) \\ & - B \delta_{ij} (1 - \delta_{XY}) \\ & - C \\ & - D d_{XY} (\delta_{j,i+1} + \delta_{j,i-1}) , \end{aligned}$$

$$I_{Xi} = + CN_e ,$$

where $\delta_{ij}=1$ if $i=j$ and 0 otherwise.

- (4) The last step is to set the initial activation value of each unit to $1/N$ plus or minus a small random perturbation (in this way the sum of the initial activations is approximately equal to N).

Questions ?

